

E4B + Serena Local Agent Bridge — Complete Setup Guide

Author: Walker Kirkpatrick

Settings and commands for running Gemma 4 E4B QAT with Serena MCP on Jetson Orin Nano 8GB

E4B + Serena Local Agent Bridge

Complete Setup Guide — Jetson Orin Nano 8GB

Date: September 22, 2026 | Author: Walker Kirkpatrick

Repository: ~/projects/e4b-serena-settings/ (GitHub: drwjkirkpatrick-web/e4b-serena-settings, push pending)

1. Hardware & Environment

Target Platform: NVIDIA Jetson Orin Nano 8GB

OS: Linux 5.15.199-tegra (aarch64)

RAM: 8GB unified LPDDR5 (CPU + GPU shared)

Power: Solar-charged Jackery Explorer 500 (100W panel, net 11.30W)

Power draw under load: 19.6W (GUI off)

Prerequisite: GUI must be OFF to free ~1.8GB RAM for the model.

1.1 Stop GUI (REQUIRED before launching model)

Command: `sudo systemctl stop gdm.service`

This frees ~1.8GB RAM. The model (4.0GB) + Serena + Python bridge need the full 8GB pool.

To restart GUI later: `sudo systemctl start gdm.service`

2. E4B Model — llama-server Launch

Model: Gemma 4 E4B QAT (gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf)

File size: 4.0GB

Location: ~/models/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf

API: OpenAI-compatible (llama.cpp built-in)

Function calling: Confirmed working (structured tool_calls JSON)

2.1 Launch Command

```
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 setsid ~/llama.cpp/build/bin/llama-server \  
-m ~/models/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf \  
--alias gemma-4-e4b-qat \  
--host 127.0.0.1 --port 8091 \  
-ngl 99 -c 32768 -ctk q4_0 -ctv q4_0 \  
-b 64 -ub 64 -fa on --jinja --fit off -np 1 \  
> /tmp/e4b_server.log 2>&1 &
```

2.2 Server Parameters — All Flags Explained

Flag	Value	Modified?	Notes
GGML_CUDA_ENABLE_UNIFIED_MEMORY	1	Kept same	Required for >4GB models on Jetson unified memory
-m (model)	gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf	Kept same	4.0GB QAT quant, UD-Q4_K_XL
--alias	gemma-4-e4b-qat	Kept same	Model name for API calls
--host	127.0.0.1	Kept same	Localhost only
--port	8091	Kept same	Dedicated E4B port
-ngl (GPU layers)	99	Kept same	All layers on GPU
-c (context)	32768	Kept same	32K context window
-ctk (KV cache type)	q4_0	Kept same	4-bit KV cache saves memory
-ctv (KV cache value)	q4_0	Kept same	4-bit KV values
-b (batch)	64	Kept same	Prompt processing batch size
-ub (ubatch)	64	Kept same	Physical batch size
-fa (flash attn)	on	Kept same	Flash attention for speed
--jinja		Kept same	Proper chat template for function calling
--fit off		MODIFIED	Added for llama.cpp build 0b1bad1+ compatibility
-np (slots)	1	Kept same	Single slot — all requests queue
setsid		MODIFIED	Detached execution — terminal wrapper times out at 420s

Note: --fit off is required by the new llama.cpp build. Without it, the server aborts when -ngl 99 is specified with an explicit context size.

Note: setsid is REQUIRED for long generations. The terminal background wrapper adds overhead and times out at 420s. E4B generating 25K tokens at ~30 tok/s takes ~833s + reasoning = ~1000-1200s per turn.

3. Python Dependencies

```
pip install mcp httpx pyyaml
```

Serena: `pip install serena-agent` (verify: `serena --version`)

llama.cpp: must be built with `GGML_CUDA=ON` for Jetson GPU

3.1 Dependency Verification

```
python3 -c "import mcp; print('mcp OK')"
```

```
python3 -c "import httpx; print('httpx OK')"
```

```
python3 -c "import yaml; print('pyyaml OK')"
```

`which serena` (should return `/home/walker/.local/bin/serena`)

4. Serena MCP Configuration

Serena version: 1.7.0

Config file: ~/.serena/serena_config.yml

Language backend: LSP (SolidLSP — pyright for Python projects)

4.1 Serena Config Settings

Setting	Value	Modified?	Notes
language_backend	LSP	Kept same	Language Server Protocol for code analysis
gui_log_window	false	Kept same	No GUI window (headless)
web_dashboard	true	Kept same	Dashboard on localhost:24282+
web_dashboard_open_on_launch	true	Kept same	Auto-opens (no display = harmless warning)
tool_timeout	240	Kept same	Per-tool execution timeout in seconds
--context	(omitted)	MODIFIED	Use DEFAULT context, NOT 'ide'
log_level	20	Kept same	INFO level (10=debug, 20=info, 30=warning)

CRITICAL: Do NOT use --context ide. The ide context excludes file tools that are essential for agentic editing: create_text_file, read_file, list_dir, find_file, execute_shell_command. Use the default context (omit --context entirely) to expose all 29 tools.

4.2 MCP Server Connection (in bridge script)

Serena is started as a subprocess via stdio transport:

command: /home/walker/.local/bin/serena

args: ["start-mcp-server", "--project", PROJECT_DIR]

Note: No --context flag (use default for all 29 tools)

5. Agent Bridge Settings

The bridge script connects E4B to Serena MCP tools via a ReAct agentic loop. Settings are in `agent_config.yaml`.

5.1 All Bridge Settings

Setting	Value	Modified?	Notes
LLM_BASE_URL	http://127.0.0.1:8091/v1	Kept same	E4B llama-server endpoint
LLM_MODEL	gemma-4-e4b-qat	Kept same	Matches --alias flag
MAX_TOKENS	25000	MODIFIED	Started at 12K, increased to 17K, then 25K for large games
TEMPERATURE	0.6	Kept same	Balanced creativity + correctness
TOP_P	0.95	Kept same	Nucleus sampling threshold
httpx timeout	6000.0s	MODIFIED	Started at 600s (failed), then 4000s, then 6000s for 25K tokens
MAX_TURNS	30	MODIFIED	Increased from 20 to 30 for complex game generation
max_tool_result_chars	8000	Kept same	Truncate long Serena tool results to keep context manageable
tool_choice	auto	Kept same	Let model decide when to call tools
Serena context	(default)	MODIFIED	Changed FROM 'ide' TO default — ide strips file tools
context_window	32768	NEW	Added for auto-trimming. Must match server -c flag. Bridge trims history to fit.

Why 6000s timeout? E4B generates at ~30 tok/s. 25K tokens = ~833s generation. With reasoning overhead (E4B thinks before acting), turns take 1000-1200s. Default httpx timeout (600s) kills the request mid-generation. 6000s gives 5x headroom.

Why 25K max_tokens? Prompt with 29 Serena tool definitions = ~13,688 tokens. E4B reasoning_content = 500-2000 tokens. Game code output = 8,000-15,000 tokens. 25K gives enough room for reasoning + complete game in one turn.

6. Context Window Management

The bridge auto-trims conversation history before each LLM call to prevent context window overflow (400 Bad Request errors).

Problem: Multi-turn conversations accumulate tokens fast. System prompt + 29 tool defs = ~14K tokens. Each E4B output = up to 25K tokens. Turn 2 input already exceeds 32K context. Without trimming, turn 4 crashes with HTTP 400.

Solution: Two-phase trimming before every LLM call:

Phase 1 — Truncate Large Message Contents

Any message content > 6000 chars is truncated to head + tail with a notice. This handles the main culprit: E4B outputs full HTML files (50K+ chars). Once written to disk by Serena, the full content does not need to stay in conversation history — a truncated preview is sufficient for context.

Phase 2 — Drop Old Middle Messages

If still over budget after Phase 1, old middle messages are dropped. Always preserved: system prompt (msg 0), user task (msg 1), and last 6 messages (recent context). A notice is inserted explaining what was trimmed. Recent window shrinks further if needed.

Token Budget Calculation

$\text{budget} = \text{context_window} - \text{tool_defs_tokens} - \text{max_output_tokens} - 500$ (safety margin)

Example: $32768 - 3500 - 25000 - 500 = 3768$ tokens available for conversation history.

Token estimation: ~4 chars per token for English/code text.

Config: context_window field in agent_config.yaml (must match server -c flag).

Verified Trimming Test

Simulated ASTRO MINER overflow (80K tokens across 5 messages) trimmed to 3.4K tokens — fits 32K context with 25K output reserve. Small conversations (under budget) are passed through unchanged.

7. System Prompt (for forcing tool use)

The system prompt combines a game-development prompt with agent instructions that FORCE the model to use Serena tools:

CRITICAL INSTRUCTIONS:

- You MUST use the `create_text_file` tool to write files. Do NOT output code as text.
- The `create_text_file` tool takes two arguments: `relative_path` (e.g. "index.html") and `content` (the full file text).
- When asked to create a game, call `create_text_file` with `relative_path="index.html"` and the complete HTML as content.
- After writing, use `read_file` to verify the file was created correctly.
- Think step by step about what tools to call.

Fallback: If E4B returns HTML as text content despite the prompt, the bridge script automatically extracts the HTML and saves it via Serena's `create_text_file` tool. This handles the common case where E4B outputs code in a markdown code block instead of calling the tool.

8. Complete Startup Procedure

Step-by-step from cold boot to running agent:

Step 1: Stop GUI

```
sudo systemctl stop gdm.service
```

Step 2: Start E4B Server

```
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 setsid \  
~/llama.cpp/build/bin/llama-server \  
-m ~/models/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf \  
--alias gemma-4-e4b-qat --host 127.0.0.1 --port 8091 \  
-ngl 99 -c 32768 -ctk q4_0 -ctv q4_0 \  
-b 64 -ub 64 -fa on --jinja --fit off -np 1 \  
> /tmp/e4b_server.log 2>&1 &
```

Step 3: Wait for Server

```
for i in $(seq 1 60); do  
curl -s http://127.0.0.1:8091/health | grep -q ok && break  
sleep 1  
done
```

Step 4: Scaffold New Project (optional)

```
cd ~/projects/e4b-serena-settings  
./new_project.sh ~/projects/my-new-game "Create a chess game"  
cd ~/projects/my-new-game  
# Edit agent_config.yaml, game_prompt.txt, system_prompt.txt as needed
```

Step 5: Launch Agent Bridge

```
cd ~/projects/my-new-game  
setsid python3 agent_bridge.py > /tmp/run.log 2>&1 &  
Or use the startup script: ./start_agent.sh
```

Step 6: Monitor Progress

```
python3 -c "import json; [print(f'[{d.get(\"type\")}] turn={d.get(\"turn\", \"?\")}'  
{d.get(\"summary\", \"\")[:200]})' for l in open('agent_log.json') for d in [json.loads(l)]]"  
Check generation slot: curl -s http://127.0.0.1:8091/slots
```

9. Gotchas (Learned the Hard Way)

1. MCP SDK uses snake_case: tool.input_schema, NOT tool.inputSchema. Pydantic throws AttributeError.
2. Serena --context ide excludes file tools (create_text_file, read_file, list_dir, find_file, execute_shell_command). Use default context.
3. E4B function calling works via llama.cpp OpenAI API — produces proper tool_calls with JSON args and finish_reason: tool_calls. No text parsing needed.
4. E4B reasoning is in reasoning_content field, separate from content. Visible in API response but not in conversation history.
5. Serena starts a web dashboard on localhost:24282+ even with no display. Harmless warning.
6. Pyright LSP auto-starts for Python projects (~3s). HTML-only projects: does nothing, does not block.
7. Tool result truncation: Serena initial_instructions returns ~15K chars. Truncate results >8K chars to keep LLM context manageable.
8. E4B may output code as text instead of calling create_text_file. Force tool use in prompt; add fallback extraction in bridge.
9. Single E4B slot (-np 1): concurrent requests queue. Do not run multiple bridges at once.
10. GPU memory does NOT release after killing llama-server on Jetson. Reboot to restart the server.
11. Context window overflow: multi-turn conversations accumulate tokens fast (14K system + 3.5K tools + 25K per turn). Bridge auto-trims before each LLM call — Phase 1 truncates large message contents, Phase 2 drops old middle messages. Configurable via context_window in YAML.

10. Verified Test Results

Four games tested with E4B + Serena on Jetson Orin Nano 8GB:

Game	Tokens	Max Tok/Turn	Time	Lines	Features	Tool Calls
Counter App (simple)	31,538	12,000	217s (3.6m)	197	13/13	3
Dungeon Crawler (17K)	44,598	17,000	1775s (29.6m)	958	24/24	1
Dungeon Crawler (12K)	18,040	12,000	950s (15.8m)	995	24/24	0 (text)
NOVA-3 Space Trader (25K)	87,551	25,000	2413s (40.2m)	1451	45/45	1
ASTRO MINER (25K)	87,493	25,000	2420s (40.3m)	1379	38/38	1

Note: The 12K dungeon crawler run produced HTML as text (no tool call), but the fallback extractor saved it. The 17K run used create_text_file directly.

Note: NOVA-3 features were 42/44 from E4B output; 3 bugs fixed manually (Galaxy class undefined, M-key map missing, CRT scanlines missing) to reach 45/45.

Note: Prior Ornith 9B single-shot FAILED on the same dungeon crawler prompt (9,367 tok, 1013s). E4B at 4B params succeeded.

Note: ASTRO MINER is the first game with a perfect score – 38/38 features, zero manual fixes needed. JS syntax valid out of the box. Hit context overflow on turn 4 (400 error) because turns 2-3 re-output the full game as text; the context trimmer was added to fix this for future runs.

Energy Analysis (solar setup)

Solar: 100W panel x 19% efficiency x 70% charge x 85% wiring = 11.30W net

Jackery Explorer 500: ~500Wh capacity

Game	Energy (Wh)	Solar Recharge	Jackery Drain	Efficiency (tok/Wh)
ASTRO MINER (25K)	13.16	69.9 min	2.63%	6,648
NOVA-3 (25K)	13.14	69.8 min	2.63%	6,663
Dungeon (17K)	9.67	51.3 min	1.93%	4,614
Counter (12K)	1.18	6.3 min	0.24%	26,645
EV Nova (Ornith 9B)	5.42	28.8 min	1.08%	1,756

E4B is 3.8x more energy-efficient than Ornith per token (6,663 vs 1,756 tok/Wh). The 4B model draws the same 19.6W but generates faster and uses fewer tokens per turn.

11. Repository Files

Location: ~/projects/e4b-serena-settings/

File	Purpose
agent_bridge.py	Configurable bridge script — reads agent_config.yaml
agent_config.yaml	All settings in one editable file
serena_e4b_bridge.py	Original hardcoded version (reference)
start_agent.sh	Universal startup: GUI stop, server check, dep verify, launch
new_project.sh	Scaffolds new projects from template
game_prompt.txt	Example game prompt (NOVA-3)
system_prompt.txt	Example system prompt for HTML game gen
README.md	Full config reference, gotchas, verified results
build_pdf.py	Custom reportlab script for this PDF (Paragraph-wrapped tables)
pdf_spec.json	JSON spec for this PDF (rebuild with build_pdf.py)
e4b-serena-setup-guide.pdf	This PDF document

Quick Start for New Projects

```
cd ~/projects/e4b-serena-settings
./new_project.sh ~/projects/my-game "Create a chess game in HTML5"
cd ~/projects/my-game
# Edit agent_config.yaml and game_prompt.txt as needed
./start_agent.sh
```